

C++ 프로그래밍 과제4

연산자 오버로딩: C++과 Python 비교분석

■ 인 적 사 항

- 학부(과): 인공지능학부 인공지능전공
- 학 번: 20230841
- 이 름: 최준수

■ 이 론 조 사

- 연산자 오버로딩이란?

연산자 오버로딩(Operator Overloading)은 객체 지향 컴퓨터 프로그래밍에서 다형성의 특정 경우로 다른 연산자들이 함수 인자를 통해서 구현을 할 때를 말한다. 연산자 오버로딩은 프로그램 개발자가 “가까운 목표 범위(Closer to the target domain)” 표기법으로 사용할 수 있고 사용자 정의 타입과 비슷한 수준을 허락하기 때문에 언어에 내장된 형식으로 구문을 지원한다.

```
1  #include <iostream>
2
3  class A{
4  public:
5      int data_a = 3;
6      int data_b = 10;
7  };
8
9  int main(){
10     A object1;
11     A object2;
12
13     std::cout << object1+object2 << std::endl; //error 발생!
14 }
```

여기 임의의 클래스 A가 있다. 이때 개발자는 A타입의 두 객체끼리 + 덧셈 연산자를 사용하여 덧셈을 하고 싶다. 하지만, 기존 C++에서 정의되어있는 + 덧셈 연산자를 이용하여 덧셈을 하려고 하면, 당연하게도 에러가 발생한다. 객체는 일반 int나 float 같은 기본 자료형처럼 단 하나의 값으로 이루어진 것이 아닌, 여러 멤버를 가지고 있기 때문이다. 여러 멤버를 가지고 있는 객체 참조 변수 object1, object2끼리 어떻게 덧셈 해야 할 지 정의되어 있지 않기 때문이다. 사전에 어떻게 덧셈을 해야 할 지 정의할 때 개발자는 연산자 오버로딩을 사용하게 된다.

개발자는 새로운 연산자를 처음부터 만들 수는 없고 Cpp에서 제공하는 기본 연산자들로만 오버로딩할 수 있다. 사용자가 오버로딩 가능한 연산자는 산술 연산자, 입출력 연산자, 단항 연산자, 비교 연산자, 증감 연산자, 첨자 연산자, 괄호 연산자, 논리 연산자, 멤버 선택 연산자(중 ->, *), 형변환 연산자, 대입 연산자, 비트 연산자, 포인터 관련 연산자, 메모리 연산자가 있다. 그중 몇 가지 연산자를 선별적으로 Python과 비교 해보도록 하겠다.

- 산술 연산자

산술 연산자는 프로그래밍에서 숫자 데이터를 계산하기 위해 사용되는 기본적인 기호로, 덧셈(+), 뺄셈(-), 곱셈(*), 나누셈(/), 나머지(%) 등이 있다. 각각 연산자를 오버로딩 하는

방법을 간단한 코드 예제로 먼저 확인하자.

```
#include <iostream>
class A{
private:
    int data;
public:
    A(int _data =0) {data =_data; }
    void getter(){
        std::cout <<data <<std::endl; }
    A operator+(const A&a2){
        return A(data +a2.data);}
    A operator-(const A&a2){
        return A(data -a2.data);}
    A operator*(const A&a2){
        return A(data *a2.data);}
    A operator/(const A&a2){
        if(a2.data !=0){
            return A(data /a2.data);}
        else{
            return A(0);}
    }
    A operator%(const A&a2){
        if(a2.data !=0){
            return A(data %a2.data);}
        else{
            return A(0);}
    }
};
int main(){
    A object1(1);
    A object2(2);
    A result;
    result =object1 +object2;
    result.getter();// 3
    result =result -object1;
    result.getter();// 2
    result =result *object2;
    result.getter();// 4
    result =result /object1;
    result.getter();// 4
    result =result %object2;
    result.getter();// 0
}
```

해당 코드에서 확인할 수 있듯, C++ 언어에서는 operator 라는 키워드를 통해 연산자를 오버로딩 할 수 있다. 클래스 내부에서 메서드 형태로 정의하는 이 기법을 ‘멤버 함수 방식’이라 한다. 반환 타입을 먼저 선언하고 operator 뒤에 오버로딩할 연산자를 명시한 후, 매개변수를 입력해 주는 방식으로 구현 가능하다.

멤버 함수 방식의 가장 큰 장점은 객체의 캡슐화를 유지할 수 있다는 것이다. 후술할 ‘비멤버 함수 방식’은 클래스 외부에서 정의되므로 캡슐화를 약화시킬 우려가 있는 반면, 멤버 함수 방식은 클래스 내부에 구현되기에 private 멤버에 직접 접근할 수 있다. 따라서 데이터의 읽기와 수정이 자유로우면서도 클래스의 경계 안에서 관리되므로 캡슐화를 해칠 가능성이 매우 낮다. 물론 상태를 변경하지 않는 경우에는 항상 const를 붙여 안정성을 높여야 한다.

그럼 이어서, 클래스 외부에서 연산자를 오버로딩하는 방식인 ‘비멤버 함수 방식’으로 구현한 산술연산자 오버로딩 코드를 확인해보자.

```

#include <iostream>
class A{
private:
    int data;
public:
    A(int _data =0) {data = _data; }
    void getter(){std::cout <<data <<std::endl; }
    friend A operator+(const A&a1, const A&a2);
    friend A operator-(const A&a1, const A&a2);
    friend A operator*(const A&a1, const A&a2);
    friend A operator/(const A&a1, const A&a2);
    friend A operator%(const A&a1, const A&a2);
};
A operator+(const A&a1, const A&a2) { return  A(a1.data +a2.data);}
A operator-(const A&a1, const A&a2) { return  A(a1.data -a2.data);}
A operator*(const A&a1, const A&a2) { return  A(a1.data *a2.data);}
A operator/(const A&a1, const A&a2) { return  A(a1.data /a2.data);}
A operator%(const A&a1, const A&a2) { return  A(a1.data %a2.data);}
int main(){
    A object1(1);
    A object2(2);
    A result =object1 +object2;
    result.getter();// 3
    result =result -object1;
    result.getter();// 2
    result =result *object2;
    result.getter();// 4
    result =result /object1;
    result.getter();// 4
    result =result %object2;
    result.getter();// 0
}

```

비멤버 함수 방식으로 구현한 산술 연산자 오버로딩은 앞선 멤버 함수 방식과 다르게 연산자 오버로딩을 구현하는 위치에서 명확한 차이를 보인다. 그렇다, operator 키워드가 클래스 밖에서 선언된 것이다. 그래서 비멤버 함수 방식은 멤버 함수 방식에는 없었던 friend 키워드를 사용했다. 멤버 함수 방식에서는 함수 자체가 이미 클래스의 멤버이기 때문에, private 멤버를 자유롭게 읽고 쓸 수 있다. 하지만, 비멤버 함수 방식으로 구현된 오버로딩 연산과 같은 클래스 외부에 정의된 함수는 기본적으로 private 영역에 접근 할 수 없다. 따라서 클래스 설계자가 이 외부 함수만큼은 내 내부 데이터에 접근 가능하다는 특별 허가를 사전에 선언해줘야한다. 그것이 바로 이 friend 키워드이다. 하지만, OOP 관점에서 보자면, 캡슐화를 약화시키는 방법이므로 해당 방식은 최소화할 필요가 있다.

또, 눈에 띄는 차이는 바로 매개변수(Parameter) 개수이다. 멤버 함수 방식에서는 연산자 오버로딩이 특정 객체가 직접 실행하는 메서드다. 즉 object1 + object2를 하면 object1이 주체가 되어 함수를 호출하므로, 자기 자신의 데이터(this->data)는 이미 알고 있다. 따라서, 피연산 데이터 (오른쪽 object2)의 데이터만 인자로 받으면 됐다. 하지만, 비멤버 함수 방식은 다르다. 이 방식은 클래스에 소속되지 않은 독립적인 전역 함수이다. 따라서, 연산을 수행할 때 사전에 선언되어 있지 않다면, 컴파일러는 어떤 데이터를 연산해야 하는지 컴파일러가 알 수 없다. 따라서, 연산자 왼쪽과 오른쪽에 위치한 두 객체를 모두 인자로 넘겨줘야한다.

```

classA:
    def __init__(self,data =0):
        self.__data =data #이름 앞 __는 C++의 private 멤버처럼 취급
    def getter(self):
        print(self.__data)
    def __add__(self,other):
        returnA(self.__data +other.__data)
    def __sub__(self,other):
        returnA(self.__data -other.__data)
    def __mul__(self,other):
        returnA(self.__data *other.__data)
    def __truediv__(self,other):
        ifother.__data !=0:
            returnA(self.__data /other.__data)
        else:
            returnA(0)
    def __mod__(self,other):
        ifother.__data !=0:
            returnA(self.__data %other.__data)
        else:
            returnA(0)
    def __radd__(self,other):
        returnA(other +self.__data)
    def __rsub__(self,other):
        returnA(other -self.__data)
    def __rmul__(self,other):
        returnA(other *self.__data)
    def __rtruediv__(self,other):
        if self.__data !=0:
            returnA(other /self.__data)
        else:
            returnA(0)
    def __rmod__(self,other):
        if self.__data !=0:
            returnA(other %self.__data)
        else:
            returnA(0)
def main():
    object1 =A(1)
    object2 =A(2)

    result =object1 +object2 # 내부적으로는 object1.__add__(object2) 호출
    result =object1 -object2
    result =object1 *object2
    result =object1 /object2
    result =result %object2
    result.getter() # 0.5
    print("--- Reflected 연산자 테스트 ---")
    r_add =10 +object1 # object1.__radd__(10) 호출
    r_add.getter() # 11

    r_sub =10 -object1 # object1.__rsub__(10) 호출
    r_sub.getter() # 9
    main()

```

앞서 살펴본 C++언어의 연산자 오버로딩은 Python 언어에서도 완벽하게 대응되는 개념이 있다. Python에서는 이를 매직 메서드라고 부르게 된다. 위 코드는 간단한 Python 언어의 매직 메서드 예제 코드이다. C++의 연산자 함수들과 Python의 매직 메서드의 대응 관계는 다음과 같다.

연산 기호	C++ 연산자 오버로딩 함수	Python 매직 메서드 (Dunder Method)	동작 설명
+	operator+(...)	__add__(self, other)	객체 간의 덧셈 연산
-	operator-(...)	__sub__(self, other)	객체 간의 뺄셈 연산
*	operator*(...)	__mul__(self, other)	객체 간의 곱셈 연산
/	operator/(...)	__truediv__(self, other)	객체 간의 나눗셈 연산
%	operator%(...)	__mod__(self, other)	객체 간의 나머지 연산

해당 코드에서는 앞서 구현한 C++ 산술 연산자 오버로딩에는 없는 새로운 연산자를 구현했다. 그건 바로 Reflected Operators이다. Reflected Operators는 Python에서 이항 연산을 수행할 때, 좌측 피연산자가 해당 연산을 처리하지 못할 경우 우측 피연산자가 대신 연산을 처리할 수 있도록 기회를 넘겨주는 특별한 매직 메서드다. 일반적으로 키워드 앞에 r을 붙은 형태이며, 그 예로 ‘__radd__’, ‘__rsub__’, ‘__rmul__’ 등이 있다.

Reflected Operator는 기본자료형과의 상호 운용성 측면 때문에 필요로 하다. 사용자가 직접 만든 클래스 객체를 파이썬의 내장 타입과 함께 자연스럽게 연산하기 위해 필수적이다. 객체 + 10은 __add__로 연산이 가능하지만, 10+객체는 int 클래스를 사용자가 수정할 수 없기 때문에 __radd__가 필수적이다. 또, 수학적 모순을 해결하기 위해서는 교환 법칙이 필연적으로 작동해야하는데, Reflected Operator 덕분에 순서가 바뀌어도 결과가 같아야 하는 연산에서 코드 작성 순서에 상관없이 동일한 결과를 출력할 수 있게 되었다.

그럼 Python의 Reflected Operator와 같이 “우측 피 연산자에게 권한을 위임하는 반사 메커니즘” 자체가 C++에 내장되어 있을까? 그렇지 않다. 하지만, 완전히 동일한 목적을 달성하기 위한 C++ 고유의 문법이 존재한다. 그것은 바로 앞서 산술 연산자 파트에서 살펴본 '비멤버 함수 방식'을 활용하는 것이다. C++은 Python처럼 연산 실패 시 우측 피연산자에게 실행 권한을 넘겨주는 동적인 위임 기능은 없다. 대신, 컴파일러가 연산자 좌우에 위치한 피연산자들의 타입을 확인하고 이에 맞는 함수를 매칭하는 정적인 방식을 취한다. 예를 들어 10 + 객체와 같은 연산을 수행할 때, 좌측 피연산자인 10은 기본 자료형이므로 클래스 멤버 함수를 가질 수 없다. Python이 이 상황에서 우측 객체의 __radd__를 호출하여 해결한다면, C++은 앞서 구현한 코드처럼 클래스 외부에 operator+(int a1, const A& a2)와 같이 양쪽 피연산자를 모두 인자로 받는 전역 함수를 정의해 두어 컴파일러가 이를 찾아 실행하도록 대응한다.

결과적으로 C++은 Python의 Reflected Operator처럼 우측 피연산자가 스스로 판단해 역순으로 연산하는 방식은 아니지만, 사전에 정의된 비멤버 전역 함수를 통해 기본 자료형과의 상호 운용성을 확보하고 코드 작성 순서에 구애받지 않는 교환 법칙을 동일하게 만족시킬 수 있는 것이다.

○ 복합 대입 연산자

복합 대입 연산자는 Compound Assignment Operators 라고 하며, 새로운 연산자가 아닌 기존 연산자에 대입 연산자를 혼합한 형태(+=, -=, *=, /=, %=)이다. 복합 대입 연산자를 좌측 피연산자(자기 자신)의 값을 직접 수정해야 하므로 클래스 내부의 멤버 함수로 구현하는 것이 일반적이다.

```

#include <iostream>
class A{
private:
    int data;
public:
    A(int _data =0) { data = _data; }
    void getter() { std::cout <<data <<std::endl; }
    // 연속된 대입 연산(예: a += b += c)을 지원하기 위해 객체 자신의 참조(A&)를 반환합니다.
    A&operator+=(const A&other) {
        this->data +=other.data;
        return *this;}
    A&operator-=(const A&other) {
        this->data -=other.data;
        return *this;}
    A&operator*=(const A&other) {
        this->data *=other.data;
        return *this;}
    A&operator/=(const A&other) {
        if(other.data !=0) {// 0으로 나누기 방지
            this->data /=other.data;}
        else{
            std::cerr <<"Error: Division by zero"<<std::endl;}
        return *this;}
    A&operator%=(const A&other) {
        if(other.data !=0) {
            this->data %=other.data;}
        return *this;}
};

```

복합 대입 연산 오버로딩은 연산 대상이 자기 자신이므로 this 포인터(this -> data)를 직접 수정할 수 있도록 클래스 내부에 멤버 함수로 작성하며, 결과는 자기 자신이 되어야 하므로 return *this;를 통해 현재 객체의 참조를 반환 gOTek. 이를 통해 (a += b) += c;와 같은 연쇄적인 연산도 가능하다.

Python 언어에서도 복합 대입 연산 오버로딩과 완벽하게 대응되는 개념이 있다. 바로 In-place 매직 메서드(__isadd__, __isub__)이다. Python에서 기본 산술 연산 매직 메서드인 __add__는 연산의 결과로 완전히 새로운 객체를 생성하여 반환한다. 반면, 복합 대입 연산자인 +=나 -=를 처리하기 위해 존재하는 __iadd__, __isub__ 등의 매직 메서드는 기존 객체(자기 자신)의 내부 상태를 직접 변경하고 반환한다는 점에서 명확한 차이를 보인다.

```

class A:
    def __init__(self, data=0):
        self.__data = data
    def getter(self):
        print(self.__data)
    # In-place 덧셈 (+)
    def __iadd__(self, other):
        self.__data += other.__data
        return self # 자기 자신(인스턴스)을 반환해야 함
    # In-place 뺄셈 (-)
    def __isub__(self, other):
        self.__data -= other.__data
        return self
def main():
    object1 = A(1)
    object2 = A(2)
    # 내부적으로 object1.__iadd__(object2) 호출
    # object1의 값이 직접 수정됨
    object1 += object2
    object1.getter() # 출력: 3
    object1 -= object2 # 내부적으로 object1.__isub__(object2) 호출
    object1.getter() # 출력: 1
main()

```

아래 코드에서 확인할 수 있듯, `__iadd__` 메서드 내부에서는 `self.__data`를 직접 수정한 뒤, 최종적으로 `return self`를 통해 자기 자신을 반환한다. 이는 파이썬에서 연쇄적인 대입을 지원하기 위한 필수적인 패턴이다.

Python의 `__iadd__`, `__isub__` 등의 In-place 매직 메서드는 C++의 복합 대입 연산자 (`operator+=`, `operator-=`)와 기능적으로 완벽하게 대응되는 개념이다. 두 언어 모두 새로운 객체를 생성하지 않고 기존 객체(자기 자신)의 내부 상태를 직접 변경한다는 점에서 동일한 메커니즘을 공유한다. 연산자의 자동 생성 여부와 관련하여, C++에서는 `operator+=`를 구현한다고 해서 `operator+`가 자동으로 생성되지 않으므로 각각 별도로 오버로딩해야 한다.

반면 Python은 `__iadd__`가 정의되어 있지 않더라도 `+=` 연산 시 차선택으로 일반 산술 연산 메서드인 `__add__`를 호출하여 연산을 수행하도록 설계되어 있다. 복합 대입 연산 오버로딩에서 `return *this`(또는 Python의 `return self`)를 통해 현재 객체의 참조를 반환하는 이유는 결과가 반드시 '자기 자신'이 되어야 하기 때문이다. 이러한 반환 처리가 이루어져야만 `(a + b) += c`와 같이 앞선 연산의 결과값을 대상으로 연달아 연산을 수행하는 연쇄적인 연산이 물리적으로 가능해진다. 즉, 자기 자신의 참조를 반환함으로써 새로운 객체 생성에 따른 오버헤드를 방지함과 동시에 대입 연산의 연속성을 보장하는 것이 이 패턴의 핵심이다.

○ 비교 연산자

비교 연산자는 두 객체의 상태를 비교하여 논리값을 반환하는 연산자로, C++에서는 `==`, `!=`, `<`, `>`, `<=`, `>=`를 오버로딩하여 사용자 정의 객체 간의 일치 여부나 대소 관계를 정의한다.

연산 기호	C++ 연산자 오버로딩	Python 매직 메서드	동작 설명
<code>==</code>	<code>operator==</code>	<code>__eq__(self, other)</code>	두 객체가 같은지 비교
<code>!=</code>	<code>operator!=</code>	<code>__ne__(self, other)</code>	두 객체가 다른지 비교
<code><</code>	<code>operator<</code>	<code>__lt__(self, other)</code>	좌항이 우항보다 작은지 비교
<code><=</code>	<code>operator<=</code>	<code>__le__(self, other)</code>	좌항이 우항보다 작거나 같은지 비교
<code>></code>	<code>operator></code>	<code>__gt__(self, other)</code>	좌항이 우항보다 큰지 비교
<code>>=</code>	<code>operator>=</code>	<code>__ge__(self, other)</code>	좌항이 우항보다 크거나 같은지 비교

C++에서 비교 연산자의 자동 생성 여부는 언어 표준 버전에 따라 명확한 차이를 보인다. C++20 이전에는 `operator==`를 정의하더라도 `operator!=`가 자동으로 생성되지 않았다. 따라서 개발자가 모든 비교 연산자를 일일이 개별적으로 오버로딩해야 하는 번거로움이 있었다. 하지만 C++20 이후부터는 '재작성(Rewriting)' 규칙이 도입되어, `operator==`만 정의하면 컴파일러가 이를 바탕으로 `!=` 연산을 수행할 때 `==`의 결과를 반전시켜 자동으로 처리한다. 특히 C++20의 3-way comparison 연산자인 `<=>` (Spaceship Operator)를 정의하면, 이를 통해 나머지 모든 비교 연산자를 컴파일러가 자동으로 생성해 줄 수 있어 구현의 편의성이 획기적으로 개선되었다. Python의 경우 역시 매우 직관적인 메커니즘을 제공한

다. 클래스 내부에 `__eq__` 메서드만 정의되어 있고 `__ne__`가 명시되어 있지 않더라도, `!=` 연산을 시도하면 파이썬 인터프리터가 자동으로 `__eq__`의 결과값을 반전(not)시켜 처리한다.

즉, 두 언어 모두 최신 표준에서는 객체의 일치 여부만 명확히 정의하면 불일치 연산은 별도의 코드 작성 없이도 논리적으로 완벽하게 동작하도록 설계되어 상호 운용성을 높이고 있다.

○ 단항 연산자

단항 연산자(Unary Operators)는 단 하나의 피연산자만을 대상으로 연산을 수행하는 기호이다. C++에서는 부호 연산자인 `+`, `-`와 비트 반전 연산자인 `~` 등을 오버로딩하여 객체 내부 데이터의 상태를 직관적으로 반전시키거나 변경할 수 있다.

C++에서 단항 연산자를 오버로딩하는 방식은 크게 멤버 함수 방식과 비멤버 함수 방식으로 나뉘지만, 일반적으로 객체 내부의 비공개 멤버에 접근하기 위해 멤버 함수 방식을 주로 사용한다. 이때 가장 중요한 특징은 매개변수가 존재하지 않는다는 점이다.

```
#include <iostream>
class A {
private:
    int data;
public:
    A(int _data = 0) : data(_data) {}
    void getter() { std::cout << data << std::endl; }
    // 단항 연산자 오버로딩 (멤버 함수 방식)
    A operator-() const { return A(-data); } // 부호 반전
    A operator+() const { return A(+data); } // 양수 유지
    A operator~() const { return A(~data); } // 비트 반전
};
int main() {
    A object1(10);
    A result = -object1; // object1.operator-() 호출
    result.getter();     // -10
    result = ~object1;   // object1.operator~() 호출
    result.getter();     // -11
    return 0;
}
```

해당 코드에서 볼 수 있듯, 단항 연산자는 호출의 주체가 되는 자기 자신 외에 별도의 인자를 받지 않는다. 이는 컴파일러가 연산자의 위치를 보고 피연산자가 하나뿐임을 인식하기 때문이다.

C++의 단항 연산자 오버로딩 역시 Python에서 완벽하게 대응되는 매직 메서드가 존재한다. Python에서는 이를 통해 사용자 정의 객체에 단항 연산을 적용할 수 있다.

연산 기호	C++ 연산자 오버로딩	Python 매직 메서드	동작 설명
-	operator-()	__neg__(self)	객체의 부호를 반전 (Negative)
+	operator+()	__pos__(self)	객체의 부호를 양수로 유지 (Positive)
~	operator~()	__invert__(self)	객체의 비트를 반전 (Bitwise NOT)

C++ 컴파일러는 동일한 operator- 키워드를 사용하더라도, 해당 연산자가 선언된 위치와 매개변수의 개수(Signature)를 통해 이항 연산(뺄셈)과 단항 연산(부호 반전)을 명확히 구분하여 처리한다. 멤버 함수 방식에서는 연산자를 호출하는 객체 자신이 이미 왼쪽 피연산자(lhs) 역할을 수행하므로, 추가로 필요한 피연산자의 개수에 따라 연산의 종류가 결정된다. 컴파일러가 이를 식별하는 기준은 다음과 같다.

구분	연산자 형태 (멤버 함수 기준)	매개변수 개수	컴파일러의 해석 방식
이항 연산자	A operator-(const A& other)	1개	obj1 - obj2문맥에서 obj1.operator-(obj2)로 호출
단항 연산자	A operator-()	0개	-obj1문맥에서 obj1.operator-()로 호출

비멤버 함수(전역 함수) 방식의 경우, 객체 외부에서 정의되므로 연산에 참여하는 모든 객체를 인자로 전달받아야 한다. 따라서 이항 연산자는 두 개의 매개변수를, 단항 연산자는 한 개의 매개변수를 가지는 방식으로 설계되어 컴파일 타임에 호출될 함수가 결정된다. 결과적으로 C++은 정적인 매개변수 타입과 개수 확인을 통해 코드의 의도를 정확히 파악하며, 개발자는 동일한 기호에 두 가지 서로 다른 의미를 부여하여 다형성을 활용할 수 있게 된다.

○ 첨자 연산자

첨자 연산자(Subscript Operator)는 배열이나 컨테이너 객체의 특정 요소에 접근할 때 사용하는 [] 기호를 의미한다. C++에서는 operator[]를 오버로딩하여 사용자 정의 클래스도 마치 내장 배열처럼 인덱스를 통해 내부 데이터에 직관적으로 접근하도록 구현할 수 있다. 먼저 간단한 구현 코드를 살펴보자.

```

#include <iostream>
class A {
private:
    int data[5] = {1, 2, 3, 4, 5};
public:
    int& operator[](int index) {
        return data[index];
    }

    const int& operator[](int index) const {
        return data[index];
    }
};
int main() {
    A obj;
    obj[0] = 10; // 쓰기 (비-const 버전 호출)
    std::cout << obj[0] << std::endl; // 읽기 (비-const 버전 호출)
    const A const_obj;
    // const_obj[0] = 10; // 컴파일 에러 발생
    std::cout << const_obj[0] << std::endl; // 읽기 (const 버전 호출)

    return 0;
}

```

해당 코드에서 확인할 수 있듯, C++에서 첨자 연산자를 오버로딩할 때는 일반적으로 const 버전과 비-const (non-const) 버전을 모두 제공해야 한다. 비-const 버전은 반환값으로 내부 데이터의 참조(Reference, &)를 반환하여 값을 읽는 것은 물론 수정하는 것(쓰기)까지 가능하게 한다. 반면, const 버전은 객체가 상수(const)로 선언되었을 때 호출되며, 내부 데이터를 수정할 수 없도록 읽기 전용으로 값을 반환한다. 만약 const 버전을 제공하지 않으면, const 객체에서 요소에 접근하려고 할 때 컴파일러가 적합한 함수를 찾지 못해 에러가 발생하게 되므로, 프로그램의 안정성을 위해 두 가지 버전을 모두 오버로딩하는 것이 필수적인 패턴이다. 이러한 C++의 첨자 연산자 역시 Python에서 완벽하게 대응되는 매직 메서드가 존재한다.

```

class A:
    def __init__(self):
        self._data = [1, 2, 3, 4, 5]
        # 값을 읽을 때 호출 (C++의 const / R-value 문맥 대응)
    def __getitem__(self, index):
        return self._data[index]
        # 값을 쓸 때 호출 (C++의 비-const / L-value 문맥 대응)
    def __setitem__(self, index, value):
        self._data[index] = value
def main():
    obj = A()
    obj[0] = 10 # __setitem__ 호출
    print(obj[0]) # __getitem__ 호출

```

Python은 C++과 달리 값을 읽고 쓰는 동작을 구분하는 방식에서 명확한 차이를 보인다. 위 코드처럼 Python은 값을 읽어올 때는 `__getitem__(self, key)`, 값을 수정하거나 할당할 때는 `__setitem__(self, key, value)`라는 두 가지 별개의 매직 메서드를 통해 읽기와 쓰기를 명시적으로 분리하여 처리한다. 그렇다면 파이썬과 달리 `__setitem__`과 같은 쓰기 전용 메서드가 분리되어 있지 않은 C++은 오직 `operator[]` 하나만으로 어떻게 읽기와 쓰기를 영리하게 구분할까? 그 해답은 바로 앞서 언급한 '참조(Reference) 반환' 메커니즘에 있다.

C++의 비-const `operator[]`는 단순한 값이 아니라 내부 데이터의 메모리 주소를 가리키는 참조형(&)을 반환한다. 따라서 `obj[0] = 10;`과 같이 대입 연산자의 좌변(L-value)에 위치할 경우, 반환된 참조 위치에 10이라는 값을 직접 쓰게 된다. 반대로 `int a = obj[0];`처

럼 우변(R-value)에 위치하면 해당 메모리 주소에 저장된 값을 자연스럽게 읽어오게 된다. 결과적으로 C++은 파이썬처럼 읽기와 쓰기 메서드를 인위적으로 분리하지 않고도, 반환된 참조값이 사용되는 문맥(L-value인지 R-value인지)에 따라 컴파일러가 알아서 읽기와 쓰기를 올바르게 수행하도록 처리한다. 이는 정적 타입 언어인 C++의 참조 특성을 극대화한 매우 고유하고 우아한 설계 방식이라 할 수 있다.

○ 함수 호출 연산자

함수 호출 연산자(Function Call Operator)는 객체를 일반 함수처럼 호출할 수 있게 만드는 operator()를 의미한다. 이를 구현한 객체를 '함수 객체(Functor)'라고 부르게 되는데, 이는 일반 함수와 달리 객체 내부에 고유한 상태(멤버 변수)를 유지할 수 있다는 강력한 장점이 존재하기 때문이다.

```
#include <iostream>
class Adder {
private:
    int total = 0;
public:
    // 함수 호출 연산자 오버로딩
    int operator()(int value) {
        total += value;
        return total;
    }
};
int main() {
    Adder add;
    std::cout << add(10) << std::endl; // 객체를 함수처럼 사용 (출력: 10)
    std::cout << add(20) << std::endl; // 내부 상태(total)가 유지됨 (출력: 30)
}
```

해당 기능은 Python의 __call__ 매직 메서드와 완벽하게 대응되는 개념이다. 클래스 내부에 __call__을 정의하면 인스턴스를 마치 함수처럼 obj() 형태로 호출할 수 있게 된다. 또한 C++의 람다(Lambda) 식 역시 컴파일러가 내부적으로 이러한 operator()를 가진 익명 함수 객체를 자동으로 생성하는 매커니즘을 취하고 있다. 따라서 람다는 함수 객체를 보다 편리하게 사용하기 위한 '구문 설탕(Syntactic Sugar)'으로 이해할 수 있다.

○ 출력 연산자와 문자열 변환

C++에서 사용자 정의 객체를 std::cout을 통해 출력하기 위해서는 operator<<를 오버로딩해야 한다. 여기서 주목할 점은 이 연산자를 멤버 함수가 아닌 '비멤버 함수(전역 함수)' 방식으로 정의해야 한다는 것이다. cout << obj 문맥에서 연산의 주체가 되는 좌측 피연산자가 사용자가 수정할 수 없는 std::ostream 타입이기 때문이다. 따라서 클래스 외부에서 함수를 정의하되, private 멤버 접근을 위해 friend 키워드를 활용하는 것이 일반적이다.

```
#include <iostream>
class A {
private:
    int data = 100;
public:
    // 연속적인 출력(Chaining)을 위해 ostream&를 반환함
    friend std::ostream& operator<<(std::ostream& os, const A& obj) {
        os << "Data Value: " << obj.data;
        return os;
    }
};
```

이러한 출력 오버로딩 방식과 Python의 `__str__`, `__repr__` 간의 공통점과 차이점은 다음과 같다.

관점	C++ operator<<	Python __str__
출력 대상	ostream인자에 의해 결정됨	호출하는 쪽(print)에서 결정됨
체이닝	cout << a << b형태의 연쇄 출력 가능	해당 사항 없음 (문자열 반환 방식)
스트림 확장성	cout, cerr, fstream등에서 범용적 동작	str() 결과를 필요한 매체에 전달함

○ bool 변환 연산자

`operator bool()`은 객체가 조건문(if, while) 내에서 논리값으로 평가되어야 할 때 사용되는 형변환 연산자이다. C++에서는 의도치 않은 암시적 형변환으로 인해 발생할 수 있는 논리적 오류를 방지하기 위해 `explicit` 키워드를 사용하는 것이 필수적인 패턴으로 자리 잡고 있다.

```
#include <iostream>
class Connection {
private:
    bool is_connected = false;
public:
    void connect() { is_connected = true; }

    // explicit을 통해 안전하고 명확한 변환을 보장함
    explicit operator bool() const {
        return is_connected;
    }
};

int main() {
    Connection conn;
    if (conn) { /* 연결 전이므로 실행되지 않음 */ }
    conn.connect();
    if (conn) { std::cout << "Connected!" << std::endl; }
}
```

해당 연산자는 Python의 `__bool__` 매직 메서드에 대응된다. Python의 경우 `__bool__`이

정의되어 있지 않더라도 `__len__`의 결과값을 참조하는 등 유연한 동적 처리를 지원하지만, C++은 `explicit` 키워드를 통해 개발자가 의도한 특정 문맥에서만 엄격하게 타입 변환이 이루어지도록 제어한다. 이는 정적 타입 언어인 C++이 지향하는 타입 안정성의 측면을 잘 보여주는 사례라고 할 수 있다.

○ C++에만 있는 연산자(Python 대응 없음)

다음 연산자들은 Python의 언어 설계 철학상 지원되지 않거나 다른 방식으로 처리된다.

C++ 연산자	Python에 없는 이유
<code>++, --</code>	Python의 정수형은 불변(Immutable) 객체이므로 직접 수치가 수정되지 않는다. 또한 <code>++x</code> 가 부호 연산자 중첩으로 오인될 수 있어 명시적인 <code>x += 1</code> 방식을 권장한다.
<code>-></code>	Python은 모든 객체를 참조로 다루며 포인터 개념을 노출하지 않는다. 따라서 객체와 멤버 사이를 구분할 필요 없이 점(.) 연산자 하나로 모든 접근을 통일한다.
<code>&&, </code>	Python은 가독성을 위해 기호 대신 <code>and</code> , <code>or</code> 키워드를 사용하며, 이는 <code>**단락 평가</code> (Short-circuit evaluation)를 보장하기 위해 연산자 오버로딩을 허용하지 않는다. 또한 모든 객체가 <code>bool</code> 컨텍스트에서 평가 가능하므로 별도의 기호 연산자가 필요하지 않다.
<code>new, delete</code>	Python은 가비지 컬렉터(GC)가 메모리를 자동으로 관리한다. 객체 생성은 클래스 호출 시 암시적으로 이루어지며, 불필요한 메모리 해제는 언어 차원에서 자동으로 수행된다.

■ 구현 코드

○ C++ : Fraction 클래스 구현

```
#include <iostream>
#include <numeric>
#include <cassert>
//Fraction (분자, 분모)
class Fraction{
public:
    Fraction(int numerator, int denominator) {
        assert(denominator !=0 &&"Error: Denominator cannot be zero!");
        num_=numerator;
        den_=denominator;
        reduce();
    }

    Fraction operator+(const Fraction&rhs) const{
        Fraction result( (num_*rhs.den_) +(rhs.num_*den_), den_*rhs.den_);
        result.reduce();
        return result;
    }

    Fraction operator-(const Fraction&rhs) const{
        Fraction result( (num_*rhs.den_) -(rhs.num_*den_), den_*rhs.den_);
        result.reduce();
        return result;
    }

    Fraction operator*(const Fraction&rhs) const{
        Fraction result((num_*rhs.num_), (den_*rhs.den_));
        result.reduce();
    }
};
```

```

        return result;
    }

    Fraction operator/(const Fraction&rhs) const{
        Fraction result((num_*rhs.den_), (den_*rhs.num_));
        result.reduce();
        return result;
    }

    Fraction operator-() const{
        return Fraction(-num_, den_);
    } // 단항 부호 반전

    Fraction&operator+=(const Fraction&rhs){
        *this = *this + rhs;
        return(*this);
    }

    Fraction&operator-=(const Fraction&rhs){
        *this = *this - rhs;
        return(*this);
    }

    bool operator==(const Fraction&rhs) const{
        return(num_*rhs.den_ == den_*rhs.num_);
    }

    bool operator!=(const Fraction&rhs) const{
        return(num_*rhs.den_ != den_*rhs.num_);
    }

    bool operator<(const Fraction&rhs) const{
        return(num_*rhs.den_) < (rhs.num_*den_);
    }

    explicit operatorbool() const{
        return(num_ != 0);
    }

    int operator[](int index) const{
        if(index == 0) return num_;
        if(index == 1) return den_;
        return 0;
    }

    friend std::ostream&operator <<(std::ostream&os, const Fraction&f);
private:
    int num_;
    int den_;
    void reduce(){
        int gcd_ = std::gcd(num_, den_);
        num_ /= gcd_;
        den_ /= gcd_;
        if(den_ < 0){
            den_ = -den_;
            num_ = -num_;
        }
    };
};

std::ostream&operator <<(std::ostream&os, const Fraction&f){
    if(f.den_ == 1){
        os << f.num_;
    } else{
        os << f.num_ << '/' << f.den_;
    }

    return os;
}

int main(){
    Fraction a(1, 2);
    Fraction b(1, 3);

```

```

std::cout <<a <<" + "<<b <<" = "<<(a+b) <<std::endl;
std::cout <<a <<" * "<<b <<" = "<<(a*b) <<std::endl;
std::cout <<a <<" == "<<Fraction(2, 4) <<" : "
        <<std::boolalpha <<(a ==Fraction(2, 4)) <<std::endl;
    if(a) std::cout <<a <<" is nonzero"<<std::endl;
    a +=b;
std::cout <<"a += b : "<<a <<std::endl;
std::cout <<"분자: "<<a[0]<<" , 분모: "<<a[1]<<std::endl;
}

```

```

C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26290.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin\code\cpp-programming\0-Homework\3.Operator Overloading in Cpp and Python\codeForHW>g++ Fraction.cpp -o Fraction

C:\Users\Admin\code\cpp-programming\0-Homework\3.Operator Overloading in Cpp and Python\codeForHW>Fraction.exe
1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
1/2 == 2/4 : true
1/2 is nonzero
a += b : 5/6
분자: 5, 분모: 6

C:\Users\Admin\code\cpp-programming\0-Homework\3.Operator Overloading in Cpp and Python\codeForHW>

```

○ Python: Fraction 클래스 구현

```

import math
class Fraction:
    def __init__(self, numerator:int, denominator:int):
        self.__num = numerator
        self.__den = denominator
    def __add__(self, other):
        result = Fraction((self.__num * other.__den )+(other.__num * self.__den), self.__den
        *other.__den)
        result.reduce()
        return result
    def __sub__(self, other):
        result = Fraction((self.__num * other.__den )-(other.__num * self.__den), self.__den
        *other.__den)
        result.reduce()
        return result
    def __mul__(self, other):
        result = Fraction((self.__num * other.__num ), (self.__den * other.__den))
        result.reduce()
        return result
    def __truediv__(self, other):
        result = Fraction((self.__num * other.__den), (self.__den * other.__num))
        result.reduce()
        return result
    def __neg__(self):
        return Fraction(-self.__num, self.__den)
    def __iadd__(self, other):

```

```

tmp =self +other
tmp.reduce()
    self.__num =tmp.__num
    self.__den =tmp.__den
    return self
def __eq__(self,other):
    return(self.__num*other.__den ==self.__den*other.__num)
def __lt__(self,other):
    return(self.__num*other.__den) <(other.__num*self.__den)
def __bool__(self):
    return(self.__num!=0)
def __getitem__(self,index): # 0: 분자, 1: 분모
    if(index ==0): return self.__num
    elif(index ==1): return self.__den
def __str__(self):
    return f"{self.__num}/{self.__den}"
def __repr__(self):
    return f"Fraction({self.__num}, {self.__den})"
def reduce(self):
common =math.gcd(self.__num, self.__den)
    self.__num //=common
    self.__den //=common
    if self.__den <0:
        self.__den =-self.__den
        self.__num =-self.__num
a =Fraction(1, 2)
b =Fraction(1, 3)
print(f"{a}{b}{a +b}")
print(f"{a}{b}{a *b}")
print(f"{a}== {Fraction(2, 4)}{a ==Fraction(2, 4)}")
ifa:
    pass
print(f"{a}is nonzero")
a +=b
print(f"a += b : {a}")
print(f"분자: {a[0]}, 분모: {a[1]}")

```

```

C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin\code\cpp-programming\0-HomeWork\3.Operator Overloading in Cpp and Python\codeForHW>python Fraction.py
1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
1/2 == 2/4 : True
1/2 is nonzero
a += b : 5/6
분자: 5, 분모: 6

C:\Users\Admin\code\cpp-programming\0-HomeWork\3.Operator Overloading in Cpp and Python\codeForHW>

```

■ 분석 및 결론

C++과 Python의 연산자 오버로딩을 직접 구현하며 비교 분석한 결과, 두 언어는 객체지향이라는 공통 분모 위에서도 서로 다른 설계 철학을 바탕으로 발전해 왔음을 확인할 수 있었다.

먼저 출력 형식 정의 측면에서 C++의 `operator<<`와 Python의 `__str__`은 객체의 문자열 표현을 목적으로 한다는 점은 같으나, 구현 방식에서 뚜렷한 차이를 보인다. C++은 스트림 객체인 `ostream`과의 상호 운용성을 극대화하기 위해 클래스 외부에서 비멤버 함수로 연산자를 정의한다. 이는 기존 스트림 라이브러리를 수정하지 않으면서도 사용자 정의 객체를 수용하고, 연쇄 출력(Chaining)을 가능케 하려는 확장성에 초점을 맞춘 설계이다. 반면 Python은 모든 데이터를 객체로 취급하는 원칙에 따라 객체 내부에 `__str__`을 정의하여 스스로의 문자열 표현을 반환하게 한다. 이는 어떤 환경에서든 객체 자체의 간결한 자기 표현을 우선시하는 설계 철학을 보여준다.

참자 연산자인 `operator[]`의 처리 방식 또한 흥미로운 차이점을 보였다. Python은 읽기 연산인 `__getitem__`과 쓰기 연산인 `__setitem__`을 명시적으로 분리하여 처리하지만, C++은 참조 반환(Reference Return) 메커니즘을 통해 단일 함수로 이를 모두 해결한다. C++의 `operator[]`가 내부 데이터의 주소값인 참조를 반환하면, 컴파일러는 해당 연산자가 사용된 문맥을 스스로 파악한다. 만약 연산자가 대입 연산자의 좌변(L-value)에 위치하면 해당 주소에 값을 직접 쓰는 동작으로 수행되고, 우변(R-value)에 위치하면 그 주소의 값을 읽어오는 동작으로 자연스럽게 연결된다. 이러한 방식은 중복된 메서드 정의 없이도 정적 타입 언어 특유의 효율성과 유연성을 동시에 확보하는 우아한 설계라 할 수 있다.

마지막으로 `Fraction` 클래스를 두 언어로 구현하며 체감한 가장 큰 차이점은 명시적 제어와 묵시적 편의성의 대조였다. C++은 `friend` 선언을 통한 캡슐화 관리나 `const` 오버로딩을 통한 타입 안정성 확보 등 개발자가 모든 상황을 엄격하게 제어해야 하는 수고로움이 따르지만, 그만큼 실행 시점의 예측 가능성이 매우 높았다. 반대로 Python은 `Reflected Operator`를 지원하거나 복합 대입 연산자 미정의 시 자동으로 산술 연산 메서드를 호출하는 등 언어 차원의 배려가 돋보여 논리적 흐름에만 집중할 수 있는 높은 생산성을 제공했다. 결론적으로 C++은 정교한 설계가 필요한 시스템 프로그래밍에, Python은 유연한 확장이 필요한 인공지능 및 데이터 처리 환경에 각각 최적화되어 있음을 깊이 이해할 수 있었다.